

Standard Operating Procedure (SOP): Web Jobs Crawling

Document Title: Web Jobs Crawling

Version: 1.1

Date: 25-Sep-25

Author: Operation Support/Platform Enhancement Naukri

1.0 Purpose and Objectives

1.1 Purpose

The purpose of this SOP is to establish a standardized procedure for crawling, extracting, validating, and processing job listings from client career websites and external job portals.

1.2 Objectives

- To systematically collect job postings from multiple web sources for analysis and reporting.
- To ensure accurate extraction of job details such as title, location, experience, skills, and application links.
- To automate data collection in a structured format, reducing manual effort and errors.
- To maintain a reliable and auditable process for monitoring new job postings and updates.

Business Impact: Effective web job crawling enables timely identification of new opportunities and market trends. By automating the extraction of job data, the team can provide actionable insights to stakeholders and reduce delays caused by manual data collection.

2.0 Scope

This SOP applies to all Technical Operations, Data Engineering, and Product team members involved in Web Jobs Crawling activities. It covers the end-to-end process of job data collection, including:

- Identification and validation of client career pages and other target job sources.
- Configuration of crawlers and extraction rules.
- Automated extraction of job details (title, description, location, experience, posted date, application link, etc.).
- Storage of crawled job data in internal systems (OMS/QC).
- Continuous monitoring, error handling, and exception management.
- Periodic quality checks, reporting, and audits to ensure accuracy and reliability.

3.0 Definitions

Term Definition

- **Web Jobs Crawling** – A feature that allows automated crawling of client career pages to identify and extract job postings. It captures key details such as job title, job description (JD), experience, location, and posted date etc.
- **OMS (Operation Management System)** – The internal system used to manage client companies, their job postings, and track all crawled job data.

- The OMS Dashboard is an internal resource used to configure, manage, and monitor client job crawling settings
- **QC Interface** – The Quality Control interface used to validate, review, and manage job publishing details before final release.
- **Company ID** – A unique identifier assigned to each client company within Naukri’s systems, used for mapping and tracking.
- **Job Field Identifier** – A unique reference to each field of a job posting (e.g., experience, location, posted date, skills) on the client’s career page, used during parsing and mapping.
- **Crawler Configuration** – The setup process defining crawl parameters such as target URLs, depth, frequency, and field mapping rules.
- **Job Parsing** – The automated process of extracting structured data (e.g., title, skills, location) from unstructured job pages.
- **Error Log** – A record of failed crawling attempts, missing fields, or data mismatches generated for debugging and quality checks.
- **Scheduling** – The mechanism for automating crawling at defined intervals (e.g., daily) to ensure job data remains updated.
- **Audit Trail** a chronological record of all actions performed during the integration process, used for verification and troubleshooting.

4.0 Types of Jobs Crawling Processes

The Web Jobs Crawling feature is designed to capture job data from multiple sources. The four crawling processes tracked in OMS are:

- **SRP Mode (Search Results Page Mode):** The crawler identifies and extracts job postings by locating XPath of key elements (e.g., job containers, headers, or anchor tags). This mode runs automatically in the backend every 5 minutes to capture updates.
- **JPERL Mode:** Jobs are crawled by matching predefined regex patterns to extract job attributes such as job title, job link, location, and job description. This mode is efficient for structured pages with predictable patterns.
- **Manual Mode:** Jobs are manually crawled when a website cannot be handled by SRP, JPERL. Manual crawling requires approval from the Crawling Tech Team and is used only as an exception.

5.0 High-Level Workflow: Web Jobs Crawling and Processing

- **Sourcing Company Data:** Company information is gathered from multiple platforms (e.g., official websites, job boards, business directories) to identify potential targets for crawling.
- **Career Page Validation:** The crawler locates and validates the client’s official career page to ensure that it contains active job listings.
- **Job Data Extraction:** Relevant job details such as job title, location, experience, job description, and application process are extracted using the configured crawling mode (SRP, JPERL or Manual).

- **Site Classification:** Target sites are classified as workable (suitable for automated crawling) or non-workable (restricted, unsupported, or structurally incompatible).
- **Exception Handling:** Errors such as missing job fields, inaccessible pages, or blocked crawlers are handled, and exceptions are updated in internal systems (OMS/QC interface).
- **Data Storage & Monitoring:** Successfully crawled job data is stored in OMS, where it can be monitored, validated, and audited for accuracy.

6.0 Detailed Jobs Crawling Procedures — SRP Mode

Phase 1: Pre-Configuration Analysis & Feasibility Check

• Identify Companies

- Access the **OMS Company List** to identify and prioritize client companies that require job crawling.
- Company IDs can be refreshed and downloaded from the OMS dashboard or obtained from the **Tech-Rectification** group on Microsoft Teams (shared by the Mapping Team).

OMS Dashboard Link:

🔗 <https://naukrics.infoedge.com:8083/#/techops/oms>

• Robot-Protection Check

Robot-protected sites cannot be crawled through any mode.

Use the following link to verify whether the career URL is robot-protected:

🔗 <http://192.168.2.123:8015/checkRobot?url=>

Paste the company career page URL into the link above to check accessibility.

• Upload Company IDs into OMS Dashboard

Pick the company IDs based on job crawling priorities.

Copy the IDs and paste them into the OMS dashboard:

1. Open
🔗 <https://naukrics.infoedge.com:8083/#/techops/oms>
2. Click on **Upload From OMS**
3. Paste the company IDs in the **OMS Data** text box in comma-separated format
Example:

4. 123456,

- **Universal Crawler (UC) Mode Validation**

When company IDs are uploaded to the OMS dashboard, also verify whether the jobs for that company are automatically crawled through **UC (Universal Crawler) Mode**.

UC Mode runs daily in the backend and fetches jobs from the client's career page.

After the crawl, the **TechOps team manually verifies** whether the job titles, locations, and JDs match the QC Interface data and the live career page.

Once the validation is confirmed, the crawler output is **manually marked as "Done"** in the OMS Dashboard.

- **Analyze Career Page**

If the client is new, open the **career page URL (CSLink)** and analyze:

- Whether job listings exist
- Number of jobs mapped by the Mapping Team
- Page structure and job distribution

This helps determine whether SRP Mode is feasible.

- **Check Career Page Type**

These types of pages usually **cannot be crawled using SRP Mode**:

- ATS systems such as **Workday, Lever, Greenhouse, Darwinbox, SuccessFactors, Oracle**, etc.
- Career pages hosted on **Cloudflare, Hostinger, Microsoft services**, or similar platforms.
- Pages where jobs are fetched through **API calls that are not using the GET method** (e.g., POST, GraphQL, token-based secure APIs).

Alternative crawling methods required include:

- **JPERL**
 - **Manual Mode**
-

- **Identify Data Structure & Build XPath**

If job details appear as plain text or static HTML, configure XPath using browser Developer Tools (Inspect Element → Console → Copy XPath).

Also, verify the **Request Method** in Network tab:

- **If the job listing API or page loads using Request Method = GET**, then the crawler *should* be able to fetch jobs through **SRP Mode**.

- After configuring SRP, check the extracted jobs in the **QC Interface (New Pool / Published Pool)**.
- **If SRP is still unable to crawl**, or no jobs appear in QC Interface after a run, *only then* move the company to **JPERL Mode** for advanced JSON/Regex-based crawling.

Example Scenarios

1) Jobs Listed in Dropdown (No Separate JD Page)

Example URL:

🔗 <https://www.provabtechnosoft.com/web-application-developer-jobs.html>

Possible XPaths:

```
//select[@name='product']/option
//select[@class='product']/option
```

(You may also identify through tag name, class, ID, etc.)

OMS Config:

- **jobInfoXPath:** (paste extracted XPath)
- **isOnlyTextSrp:** true

Because Title, Experience, Location, JD appear on the same page

2) Jobs with JD on a Different Page (Single Job Details Page)

Example URL:

🔗 <https://www.factsoftware.com/careers>

Possible XPath:

```
//a[@class='button-4 w-button']
```

OMS Config:

- **jobInfoXPath:** (paste this XPath)
- **isOnlyTextSrp:** false

Because JD opens on a separate page

Also configure:

- **Next / Load More XPaths**
- **JD Extraction Rule Set**

- Any pagination or expandable job listing controls

After updating configurations, click **Close**, and the system automatically saves the SRP config.

3) Jobs with Title, JD, Location, and Other Details on the Same Page

Example URL:

🔗 <https://ivserve.com/career>

In this scenario, all job information (Title, JD, Experience, Location, Job Responsibilities, etc.) appears directly on the same page without navigating to a separate JD page.

XPath Example for job container:

```
//div[@class="hiring_services"]
```

OMS Config:

- jobInfoXPath: (paste the above XPath)
- isOnlyTextSrp: **true**
(because Title, JD, Location, and all details are displayed on the same page in a single structured block)

• Final Configuration in OMS

After XPath and behavior rules are added:

1. Select:
 - **crawlerType:** SRPAUTOMATION
 - **siteType:** SRP
2. Paste the client **Career Page URL**
3. Add internal comments such as:
 - *No JD available*
 - *Robot Protected*
 - *No Jobs on Career Page*
 - *Manually Done*
 - *XPath Exception*
 - *Next button not clickable, etc.*

These comments help TechOps & Mapping teams in tracking crawler outcomes.

Phase 2: Crawling Configuration & Execution – SRP Mode

• Configure Crawlers

Based on the feasibility assessment from **Phase 1**, configure the crawling mode in the OMS dashboard. This includes selecting the SRP Mode, defining XPath's, and ensuring all required parameters are correctly mapped.

• SRP Mode (Search Results Page Mode)

- In **SRP Mode**, the crawler identifies job postings by targeting the XPath of list containers such as <div>, , headers, or anchor tags <a> representing individual job cards.
- The crawler runs *automatically in the backend every 5 minutes*.
- When the config shows:
"xpath": "", "threshold": 100

It means that the SRP crawler triggers periodically, scanning up to **100 job entries** per run to ensure frequent updates and immediate detection of newly posted jobs.

• Validation of Crawled Jobs

After every SRP run:

- Job titles, job URLs, and other extracted fields must be validated in the **QC Interface**:

🔗 **QC Dashboard:**

<http://crawling-webjobs.infoedge.com/web/index.php/Job/index>

This ensures that job listings have been extracted accurately and consistently.

• Execution Monitoring

- Continuously monitor crawler logs for any failures, exceptions, junk data, or XPath breakdowns.
 - Update or refine the XPath rules as required before enabling the crawler for scheduled, automated runs.
 - Ensure Next-button, Load More, and JD extraction rules are functioning end-to-end.
-

Phase 3: Local Testing & Job Validation

• QC Interface Verification

After the SRP crawler extracts jobs, review the outputs in the QC Interface. Jobs appear in three major pools:

1. **New/Modified Jobs Pool** – Recently crawled or updated jobs
2. **Published Jobs Pool** – Verified and active job listings

3. **Hold Jobs Pool** – Jobs requiring manual verification or temporarily disabled due to issues

- **Job Content Validation**

Inspect all entries in the **New/Modified Jobs** pool:

- Remove any **junk jobs**, mismatched job titles, or irrelevant postings.
 - Identify and eliminate **duplicate entries**.
 - Validate job titles, locations, and JD accuracy against the client's career page.
 - Move incorrect or invalid jobs into the **Hold Jobs Pool**.
 - Only publish jobs that accurately represent the live listings on the client's site.
-

- **Completion & Record Keeping**

- Once validation is completed, update the OMS Dashboard and mark the job crawling task as **"Done"**. This ensures proper documentation, tracking, and transparency for future audits and team reference.

7.0 Detailed Jobs Crawling – JPERL Mode

1. What is JPERL?

JPERL Mode is a backend crawling mechanism used to extract jobs from client career pages when:

- Jobs are not easily accessible via simple HTML/XPath (SRP Mode)
- Data is rendered through **JavaScript, JSON APIs, or embedded script blocks**
- The page requires **regex, JSON parsing, or special headers/POST body** to fetch job data

In JPERL, you configure **rules and patterns** instead of writing code. The crawler then:

1. Fetches HTML / JSON from the page or API
 2. Applies **JSON mappings or regex** to extract:
 - Job ID
 - Job Title
 - Location
 - Job Link
 - Posted Date
 - Job Description
 3. Saves the jobs into the internal database
-

2. When to Use JPERL (Feasibility & Decision)

You choose **JPERL** when **SRP Mode is not suitable**, for example:

🔗 Use JPERL for:

- **ATS systems** like:
 - Workday, Lever, Greenhouse, Darwinbox, SuccessFactors, Oracle, BambooHR, Ceipal, Keka, etc.
- **JS-heavy / API-driven sites:**
 - Where job data loads via XHR/API calls
- **JSON-based or script-embedded job data**
- **Sites hosted on platforms like:**
 - Cloudflare, Hostinger, Microsoft services, etc.
- **Plain HTML sites where SRP cannot detect stable job blocks**

🔗 When NOT to use JPERL:

- Page is **Robot-blocked**
- No job content is accessible
- Completely static brochure sites with no job listing

3. Inputs & Sources – Where You Start

3.1 Get Company IDs

You normally pick companies for JPERL when SRP fails or is not possible.

- From **OMS Company List**
- From **OMS Dashboard**
- From **Tech-Rectification group on MS Teams** (Mapping team shares new client IDs)

🔗 OMS JPERL Dashboard

<https://naukrics.infoedge.com:8083/#/techops/perl>

3.2 Robot Check (Compulsory Before Config)

Some sites are blocked via robots.txt. Those **cannot be crawled** by any mode (SRP/JPERL/UC).

Use internal Robot Checker:

🔗 <http://192.168.2.123:8015/checkRobot?url=>

- Paste **career page URL**
- If response or log shows:
- Response Body returned by robots.txt checker: 2

→ **Site is robot-protected** → mark as **ROBOT** in OMS → do not configure JPERL.

4. Analyzing the Career Page (Tech Analysis Steps)

Before writing any JPERL config:

1. **Open career page in browser**
2. Press F12 → open **Developer Tools**
3. Go to **Network** tab
4. Refresh the page

Now look for:

- **XHR / Fetch / API calls** returning JSON (e.g., jobs list)
- **HTML** containing <div>, , <a> job cards
- **<script> tags** that contain JSON job arrays
- **Pagination patterns** (page=1, offset=20, etc.)
- Whether **JD is on same page** or **separate JD URL**

Try to answer:

- Is job data in **JSON**?
 - Is job data only in **HTML**?
 - Is there **pagination**? How?
 - Is JD in the same document or a **new URL**?
-

5. Tools You Use with JPERL

5.1 Browser Developer Tools

- Check **network calls**
 - See response data (JSON/HTML)
 - Inspect **headers & request payload**
-

5.2 Regex Tool – regex101

🔗 <https://regex101.com/>

Use this to:

- Write and test regex for:
 - Job blocks
 - JD segments

- IDs, titles, locations, etc.

Example regex Pattern:

```
(?s)"id":([^\,]+).+?location":"([^\"]+).+?designation":"([^\"]+)[^\"]+"qualification":"([^\"]+).+?description":"(.+?)</p>"}
```

This regex is designed to extract **multiple job fields** from a **JSON-like or HTML-embedded JSON block**.

Below is a complete explanation of every part.

1. (?s)

This is a **DOTALL flag**.

- ✓ Makes the dot . match **newline characters** also
- ✓ Needed because JSON or HTML may span multiple lines

Without (?s), the regex would break if the job content is multi-line.

2. "id":([^\,]+)

This matches:

- Literally "id":
- Then captures everything **until the next comma**

🔍 **Captured as Group 1 → JOB ID**

Example:

"id":12345,

Captured → 12345

3. .+?location":"([^\"]+)

- .+? = non-greedy match (skips all text until next match)
- It finds **location":"**
- Captures everything until the next "

🔍 **Captured as Group 2 → LOCATION**

Example:

location":"Bangalore"

Captured → Bangalore

4. `.+?designation":("[^"])+`

- Jumps to the next occurrence of `designation":`
- Captures everything until the next `"`

🔍 Group 3 → Job Title / Designation

Example:

designation":"Software Engineer"

Captured → Software Engineer

5. `.+?experience":("[^"])+`

Extracts the experience field.

🔍 Group 4 → Experience

Example:

experience":"2-5 Years"

Captured → 2-5 Years

6. `[^"]+"[^"]+"qualification":("[^"])+`

This part skips some unwanted values using:

- `[^"]+" → skip until next ", then skip another field`

Then matches:

qualification":"..."

🔍 Group 5 → Qualification

Example:

qualification":"B.Tech / M.Tech"

Captured → B.Tech / M.Tech

7. `..+?description":"(.+?)</p>"}`

This extracts the **Job Description (JD)**.

- `description":"` — starting point
- `(.+?)` — non-greedy capture **until**
- `</p>"}` — closing tag

🔗 Group 6 → Job Description

Example:

`description":"We are hiring engineers...</p>"}`

Captured → We are hiring engineers...

Summary of What the Regex Extracts

Group	Field Extracted
1	Job ID
2	Location
3	Designation / Job Title
4	Experience
5	Qualification
6	Job Description (JD)

5.3 JSON Formatter – jsonformatter

🔗 <https://jsonformatter.org/>

Use to:

- Beautify JSON
- Understand nested structure
- Identify keys like:
 - id
 - title
 - location
 - description

5.4 HTML Viewer – [htmledit.squarefree](https://htmledit.squarefree.com/)

🔗 <https://htmledit.squarefree.com/>

Use this to:

- Paste JD HTML
- Visualize tags
- See <p>, , , etc.
- Clean up complex/nested HTML
- Decide which tag to capture via regex for JD

5.5 Sublime Text

Use Sublime to:

Trim Unwanted Text

Career pages often include:

- HTML comments (<!-- -->)
- Unused JavaScript
- Extra CSS blocks
- Tracking scripts
- Unnecessary whitespace

Sublime allows easy trimming using:

- **Find → Replace**
- **Regular Expressions**
- **Multi-line selection**

Example: Remove all comments

Find:

<!--.*?-->

Replace: *(empty)*

5.6 curlconverter + Postman

- <https://curlconverter.com/> – converts browser network requests to cURL/Python/Postman
- **Postman** – test APIs with headers + POST body

Use when:

- Site needs **headers, cookies, tokens, or POST data** to return jobs.

6. JPERL Configuration Concepts

This is the **brain** of JPERL crawling.

You configure key–value pairs like:

- Where to find jobs
- How to parse them (JSON or regex)
- How to extract JD
- How to paginate
- Whether to use proxy
- How to format dates

Below is a dictionary of the main parameters.

6.1 LOCJSON & LOCJSONSEQ – JSON-Based Job List

When to use:

Job list is in JSON, e.g.:

```
{
  "jobs": [
    {
      "id": "123",
      "title": "Software Engineer",
      "location": "Bangalore",
      "description": "<p>JD here</p>"
    }
  ]
}
```

Parameters:

- LOCJSON – path in JSON or how data is accessed (internal config)
- LOCJSONSEQ – list of fields to pick, in order

Example:

LOCJONSEQ = JOBID,JOBTITLE,JOBLINK,LOCATION,JOBDESC

JPERL expects your JSON extraction to map each key to these fields.

6.2 LOCRGX & LOCRGXSEQ – Regex-Based Job List

When to use:

Job list is in HTML, not JSON. For example:

```
<div class="card card-job">
  <a href="/careers/123">Senior Engineer</a>
  <span class="location">Bangalore</span>
</div>
```

Regex Example (from your Riyo Advertising config):

```
(?s)<div class=\"card card-
job\">[^\"]+\"[^\"]+\"[^\"]+\"([^\"]+)\">[<]+>[<]+>([<]+)<[<]+>[<]+>[<]+>([<]+)<
```

This might extract (as capturing groups):

1. JOBID or internal key
2. JOBLINK or related value
3. JOBTITLE
4. LOCATION

Then you define:

LOCRGXSEQ = JOBID,JOBLINK,JOBTITLE,LOCATION

So the crawler knows which group is which field.

6.3 JDJSON1 & JDJSONSEQ1 – JD from JSON

When to use:

- JD is in a separate JSON object/endpoint
- Job list is in LOCJSON, but JD is only available via JDJSON

Example:

JDJSON1 – JSON that contains "description": "..."

JDJSONSEQ1 = JOBDESC

6.4 JDRGX1, JDRGXSEQ1 – JD from HTML via Regex

Used when JD is stored in HTML sections.

Example from your config:

```
"JDRGX1": "(?s)<h1>Job Title:(.*?)contact-info\\>",
"JDRGXSEQ1": "JOBDESC"
```

This regex will capture all content between <h1>Job Title: and contact-info"> as JD.

You can define multiple JD regex patterns:

- JDRGX1 / JDRGXSEQ1
- JDRGX2 / JDRGXSEQ2

...to support different page layouts.

6.5 ESCAPE_VARJOBLINK

Sometimes JSON job links look like:

```
"https:\\\\company.com\\careers\\job\\123"
```

With ESCAPE_VARJOBLINK = No, JPERL converts this to:

```
https://company.com/careers/job/123
```

So that JOBLINK becomes a valid URL.

6.6 Pagination – !0o!CURPG!0o! and !0o!STARTJOBNO!0o!

Page-based pagination:

URLs like:

```
?page=0
?page=1
?page=2
```

Use:

```
!0o!CURPG!0o!
```

Example:

URL = <https://site.com/careers?page=!0o!CURPG!0o!>

Offset-based pagination:

URLs like:

offset=0
offset=10
offset=20

Use:

!0o!STARTJOBNO!0o!

Also set:

"MAXPAGESPARSE": 10

So JPERL knows how many offset jumps to make.

6.7 Multiple URL Inputs – !0o!URLVARS!0o! and URL_VARS

Use Case:

Different locations or departments require different URLs.

Example:

URL_VARS = bangalore!@@!noida!@@!hyderabad
URL = <https://site.com/jobs?location=!0o!URLVARS!0o!>

Crawler will run the same logic for each location.

6.8 PROXY = "Yes"

Use when:

- Site is blocked on server
- Logs show 403/503 or no response
- Client is accessible only via proxy

Be careful – proxy usage consumes quota.

6.9 JDVARS – Classifiers

From your example:

```
"JDVARS": {
  "FAREA": { "callable": "Classifier_FAREA", "type": "SUB" },
  "INDUSTRY": { "callable": "Classifier_INDUSTRY", "type": "SUB" },
  "ROLE": { "callable": "Classifier_ROLE", "type": "SUB" }
}
```

These are internal classifiers that run on the JD text to classify:

- Functional Area
- Industry
- Role

6.10 Other Common Fields

- CanProceed: flag to allow execution
- DELETE_UNLISTED: "y" → delete jobs not found this run
- MOVE_TO_JD: 1 → indicates JD comes from separate page
- JOBLINK: usually uses {{VARJOBLINK}}
- SQLDATE: date format, e.g. %Y-%m-%d
- USER_AGENT: e.g. "Firefox/5.0"
- PROXY: "Yes" for proxy usage
- POSTQUERY: SQL to update comp name/ID, etc., after crawl completes

7. Predefined ATS Rules (Very Important)

For some ATS platforms, **JPERL rules are already predefined** in backend.
You usually **only change**:

- URL (main URL)
- URL_VARS (company name / domain code)

No need to write regex or JSON again.

Some of these include:

- Greenhouse → <https://boards.greenhouse.io/<company>>
- Lever → <https://jobs.lever.co/<company>>
- BambooHR → [.<company>.bamboohr.com/careers](https://<company>.bamboohr.com/careers)
- Ceipal / Cieple
- Keka HR
- and similar ATS

That's why for those ATS types:

You mostly just plug in URL and URL_VARS, rest is handled by predefined JPERL templates.

8. End-to-End Example – Riyo Advertising (4014506_SRP)

Here's the **full example config you shared**, annotated conceptually.

```
{
  "4014506_SRP": {
    "CanProceed": 1,
    "Cookie": "YES",
    "CurrentJobNum": 1,
    "CurrentPageNum": 0,
    "DELETE_UNLISTED": "y",
    "ISCRAWL_FULL": 0,
    "IS_SITE_ID_DERIVED": 0,

    "JDRGX1": "(?s)<h1>Job Title:(.*?)contact-info\\>",
    "JDRGXSEQ1": "JOBDESC",

    "JDVARS": {
      "FAREA": { "callable": "Classifier_FAREA", "type": "SUB" },
      "INDUSTRY": { "callable": "Classifier_INDUSTRY", "type": "SUB" },
      "ROLE": { "callable": "Classifier_ROLE", "type": "SUB" }
    },

    "JOBLINK": "{{VARJOBLINK}}",
    "JOBSCOUNT": 0,

    "LOCRGX": "(?s)<div class=\\\\"card card-job\\\\">[\\^"]+\\\\"[\\^"]+\\\\"[\\^"]+\\\\"(((\\^\\")+))\\\\">[\\^>]+>[\\^>]+>([\\^<]+)<[\\^>]+>[\\^>]+>[\\^>]+>([\\^<]+)<",
    "LOCRGXSEQ": "JOBID,JOBLINK,JOBTITLE,LOCATION",

    "MAXJBSPPAGE": 10,
    "MAXPAGESPARSE": "1",

    "MOVE_TO_JD": 1,
    "POSTQUERY": "update WEB_JOBS set COMPNAME ='Riyo Advertising',compid ='4014506', jobConsultant
= 'n' where SITE = '4014506_SRP'",

    "PROXY": "Yes",
    "SAVE_JD_COOKIE": "Yes",

    "SQLDATE": "%Y-%m-%d",
    "StartPageNum": 0,

    "TERMRGX": "",
    "URL": "https://riyoadvertising.com/CurrentOpening.shtml",
```

```
"USER_AGENT": "Firefox/5.0"
}
}
```

What this configuration does:

- Hits <https://riyoadvertising.com/CurrentOpening.shtml>
- Uses LOCRGX to extract:
 - JOBID
 - JOBLINK
 - JOBTITLE
 - LOCATION
- For each job, opens JOBLINK (because MOVE_TO_JD = 1)
- Uses JDRGX1 to extract JD between Job Title: and contact-info">
- Runs JDVARS classifiers on JD
- Marks jobs with compname = Riyo Advertising & compid = 4014506
- Uses PROXY because site is blocked on server

9. Running JPERL & Checking Logs

9.1 After Creating Config – Hit RUN

Then check NIC logs:

http://192.168.2.123/nicrawledlogs/crawled/YYYY-MM-DD/siteID_UC/siteID_SRP.html

Replace:

- YYYY-MM-DD with today's date
- siteID_SRP with e.g., 4014506_SRP

In logs, check:

- Job count found
- Any regex errors
- Any robot message
- Any 403/503
- JSON parse errors
- Whether JD extraction worked

10. QC & Validation Flow (After JPERL Crawl)

Once backend crawl is done (usually next morning):

1. Open **QC Interface**

2. Check:
 - **New/Modified Jobs Pool**
 - **Published Jobs Pool**
 - **Hold Jobs Pool**
3. Validate:
 - Title
 - Location
 - JD accuracy
 - Links open correctly
 - No duplicate or junk jobs
4. Move invalid jobs → **Hold**
5. Publish valid jobs
6. Mark job crawling as **Done** in dashboard

JPERL Issue Escalation & Tracking Process

When Jobs Do Not Appear After JPERL Crawl

If the crawler does not fetch jobs the next morning (missing jobs, zero jobs, partial JD, broken regex, or JSON not loading), follow the mandatory escalation workflow below.

1. Log Issue in the Official JPERL Issues Sheet

All JPERL extraction failures must be logged in the central tracking sheet:

JPERL Issues Sheet (SharePoint)

 https://infoedge-my.sharepoint.com/:x/r/personal/samyak_srivastava_infoedge_com/_layouts/15/Doc.aspx?sourcedoc=%7B81FFCF4D-30AA-4674-8DE8-407E7879EEFE%7D&file=JPerl%20Issues.xlsx&action=default&mobileredirect=true

2. Fill the Issue Details in the Following Format

Field	Description
Date	Date when the issue is noticed
siteID	The JPERL site ID (e.g., 4014506_SRP)
Issue	Describe the exact problem (e.g., <i>0 jobs fetched, JD missing, regex mismatch, 404 error, JSON blank, Request blocked</i>)
Interface	Mention where the issue is visible (<i>QC Interface, Logs, Career Page</i>)
Issue Raised By	Your name / TechOps executive reporting
Expected Jobcount	Job count visible on the client's career site
Issue Cause	Write the suspected cause if known (<i>regex broken, structure changed, need proxy, API blocked, robot issue, etc.</i>)

This helps backend engineers quickly diagnose and fix the problem.

3. Mention Likely Cause (If Known)

While logging the issue, include one of the common failure causes:

- **JSON structure changed** (keys modified / nested JSON)
- **Regex mismatch** due to UI redesign
- **JD API endpoint changed or removed**
- **CORS blocked / server-side restrictions**
- **403 / 503 errors in logs**
- **Robots.txt disallows crawling**
- **Proxy required** (site blocked on server)
- **POST request needs headers / dynamic parameters**
- **Career page temporarily down**
- **Pagination changed (offset/page/limit changed)**

Even if unsure, providing hints helps the backend team troubleshoot faster.

4. Proxy Usage Rule (IMPORTANT)

- **Some client sites block requests originating from the server.**
- **In such cases, PROXY = Yes must be added in JPERL configuration.**
- **However, proxy consumes paid usage, so:**

Proxy should be enabled ONLY after approval from Backend Tech Team.

Never use proxy without explicit technical approval.

This prevents unnecessary proxy consumption and ensures controlled usage.

5. Backend Team Responsibilities After Issue is Logged

Once the issue is added to the sheet:

- Tech Team checks logs & crawler behaviour
- Reviews **JSON / regex / URL / headers**
- Updates configuration if required
- Enables **proxy** only if necessary
- Reruns JPERL crawler
- Adds remarks in the Issues Sheet
- Hands back configuration to TechOps for validation

6. If Still Not Resolved → Perform Manual Crawling

If the backend team confirms the site cannot be crawled due to:

- Strong robot protection
- Encrypted JS data
- Cloudflare / Hostinger blocking
- Captcha walls
- Closed API
- ATS restrictions
- Dynamic rendering not detectable by crawler

Then the crawling must be done in Manual Mode.

Add remark in OMS:

"JPERL not feasible – switched to Manual Crawling."

8.0 Detailed Jobs Crawling Procedures – Manual Mode

Manual Mode is used when jobs **cannot be crawled via SRP, JPERL**, either because the site blocks automated crawling or the page structure is too complex. This method relies on **manual extraction and entry** of job data.

Phase 1: Pre-Configuration Analysis & Feasibility Check – Manual Mode

- **Identify Companies:** Access the OMS Company List to determine which client sites require manual crawling.
- **Feasibility:**
 - Sites that **block crawlers** or **cannot be crawled through any automated mode (SRP, JPERL)** are candidates for Manual Mode.
 - **Robot-protected sites** cannot be crawled through any mode.
 - Manual crawling requires **approval from the Crawling Tech Team** before starting.
 - Currently, platforms such as **Darwin** and **Notion** are approved for manual posting.

Phase 2: Manual Job Extraction & Entry

- **Data Collection:**
 - Manually copy job details from the client's career page, including:
 - Job Title
 - Job Link
 - Location

- Experience
- Job Description (JD)
- Company Id
- Company Name
- Min Sal
- Max Sal
- Posted Date
- **Data Entry:**
 - Enter the collected information into the **WJPA (Web Jobs Posting) Sheet**.
 - Verify the accuracy of all fields before submission.
- **Submission for Listing:**
 - After validation, use the “**Add Offline**” button in the WJPA sheet to forward jobs to the listing team for processing.

Phase 3: QC Verification & Record Keeping – Manual Mode

- **WJPA Sheet Verification:**
 - Before sending jobs to the listing team, verify all manually entered data in the **WJPA sheet** for accuracy, completeness, and formatting.
 - Ensure fields such as **Job Title, Job Link, Location, Experience, and JD** are correctly filled.
- **Submission & Blocking Crawling Mode:**
 - After verification, use the “**Add Offline**” button in the WJPA sheet to forward jobs to the listing team.
 - Once jobs are processed and added to **OMS**, mark the site in the **QC Interface** to **block automated crawling modes** for that client.
- **Record Keeping:**
 - Maintain an **audit trail** of all manual postings, including the source URL, date, and person responsible.

10.0 Post-Go-Live Jobs Monitoring & Troubleshooting

Daily Monitoring:

- Verify that newly crawled or manually added jobs appear correctly in the **QC Interface** or **WJPA sheet**.
- Check that **New/Modified Jobs Pool** and **Published Jobs Pool**, reflect accurate job statuses.

- For Manual Mode, confirm that verified WJPA entries are successfully processed into OMS.
- For jobs crawled from **ATS or client sites**, compare the **previous day's jobs** with the **current pool**.
- If jobs **match exactly**, keep them in the **New/Modified Jobs Pool**.
- If jobs are **junk, outdated, or extraneous**, move them to the **Hold Jobs Pool**.

Purpose:

- Ensures only valid, live jobs are retained in the active pool.
- Helps maintain clean job data for OMS and QC tracking.

Integration:

- This step should be performed **daily as part of post-go-live monitoring** across all crawling modes (SRP, JPERL, Manual).

11.0 Troubleshooting Failures:

Initial Investigation:

- If jobs fail to appear in the **New/Modified Jobs Pool**, first check the **crawler configuration** and the **career page structure**.
- For **JPERL or SRP**, review **regex patterns, JSON key-value mappings, XPath locators, or job links**.

Configuration Updates:

- Modify the configuration as needed to match the current **page structure**.
- Ensure headers, encoding, and request parameters are correctly set if the URL alone is insufficient.

Escalation to Tech Team:

- If the configuration is correct but jobs still do not appear in the **New/Modified Jobs Pool**, escalate the issue to the **Crawling Tech Team**.
- Document the issue in the **SharePoint "JPERL Issue Sheet"**, including:
 - Type of issue faced
 - Number of affected jobs
 - Date of occurrence
 - Client/site information
 - Any other relevant details for troubleshooting

Follow-Up & Resolution:

- Track the resolution progress with the tech team.
- Once resolved, verify that jobs are correctly populated in the **New/Modified Jobs Pool**.
- Update the SharePoint sheet with resolution details and any corrective actions taken.

12.0 Audit Process for Quality Assurance

To ensure the integrity, accuracy, and completeness of job data crawled via SRP, JPERL and Manual Modes, and to maintain compliance with internal quality standards.

13.0 Pre-Go-Live Audit (For every new integration)

- **Objective:** To catch configuration errors and validate the crawling setup before the integration is activated for live job extraction.

Process:

1. Peer Review:

- A second engineer, **not involved in the initial configuration**, must review the entire crawling setup for the client.

2. Checklist Verification: The reviewer will verify:

- **Crawling Configuration:** Ensure URL, regex patterns (for HTML/JS), JSON key-value mappings.
- **Predefined Parameters:** Verify headers, encoding, and request parameters (if used) are correctly configured.
- **Data Fields:** Confirm all critical job fields (Job Title, Job Link, Location, Experience, JD) are correctly mapped in the configuration.
- **Manual Mode Approval:** Ensure sites requiring manual crawling have tech team approval.
- **Testing:** Perform a **test crawl** on a few sample jobs to verify that data is extracted and stored correctly in OMS or WJPA sheet.

Outcome:

The crawling integration **cannot go live** without a successful Pre-Go-Live Audit and **sign-off from the reviewing engineer**.

14.0 Audit Trail & Documentation

- **Documentation Requirement:**
 - All audit and monitoring activities for web jobs crawling must be documented in the **Crawling Dashboard** or equivalent internal tracking system.
- **Audit Record Details: The documentation should include:**
 - **Date of Audit:** When the audit or monitoring was performed.
 - **Auditor Name:** Engineer or team member performing the audit.
 - **Company ID:** Identifier of the client company being audited.
 - **Findings & Actions Taken:** Details of any issues discovered and corrective actions performed (e.g., "Regex pattern updated due to change in job link structure").
 - **Final Status:** Pass/Fail or Verified/Blocked depending on the outcome of the audit.
- **Purpose:**
 - To maintain a **clear record of all crawling activities**, configuration changes, and quality checks.

- Ensures accountability, traceability, and compliance for all automated and manual job extraction processes.

15.0 Useful Links & References

The following internal resources are referenced throughout the Web Jobs Crawling process:

1. **Robot Check:**
 - Verify if a site blocks crawlers before configuring automated crawling.
 - URL: <http://192.168.2.123:8015/checkRobot?url=>
2. **OMS Dashboards:**
 - SRP Mode Dashboard: [OMS SRP Dashboard](#)
 - JPERL Mode Dashboard: [OMS JPERL Dashboard](#)
3. **QC Interface:**
 - For validating and managing crawled job data.
 - URL: QC Interface
4. **JPERL Issue Tracking:**
 - Document JPERL extraction or configuration issues.
 - URL: [JPERL Issue Sheet](#)
5. **Manual Mode Reference:**
 - WJPA offline Excel micro file for manual job entry and verification.

Document Approval

Prepared By: Rohit Kumar

Date: 26-Nov-25

Reviewed By: _____

Date: _____

Approved By: _____

Date: _____